

Министерство образования и науки Российской Федерации
Санкт-Петербургский государственный политехнический университет

—
Институт компьютерных наук и кибербезопасности
Высшая школа кибербезопасности

**ОТЧЕТ
ПО ЛАБОРАТОРНОЙ РАБОТЕ № 5**

«Оптимизация запросов и производительность»

по дисциплине «Системы управления базами данных»

Выполнил
студент гр. 5131001/20502

<подпись>

Улановский Г. А.

студент гр. 5151001/20202

<подпись>

Игнатьев И.В.

Проверил
ассистент

<подпись>

Пономарев Д. А.

Санкт-Петербург
2026

СОДЕРЖАНИЕ

1	Формулировка задания	4
2	Теоретические сведения.....	5
2.1	Планы выполнения запросов	5
2.2	Индексы и оптимизация запросов	5
3	Результаты работы	7
3.1	Подготовка экспериментальной базы данных	7
3.2	Заполнение тестовыми данными	8
3.3	Запрос Q1: выборка секретных действующих документов	9
3.4	Запрос Q2: агрегация операций сотрудника за период	11
3.5	Запрос Q3: поиск документов с изменением срока действия.....	13
3.6	Сводные результаты и графики	15
4	Ответы на контрольные вопросы	19
4.1	Что такое логический план выполнения запроса, когда он формируется и для чего используется?	19
4.2	Как связаны логический и физический план выполнения запроса?.....	19
4.3	Что такое предварительный и реальный план выполнения запроса? Почему они могут отличаться?	19
4.4	Как селективность атрибута связана с индексами и производительностью?	20
4.5	Когда применение индексов эффективно?	20
4.6	Как порядок полей в составном индексе влияет на производительность различных запросов и применение индекса?	20
4.7	Какая операция указывается в физическом плане запроса?	20
4.8	Как прочитать текстовый план выполнения запроса от первой операции к последней?	21
4.9	Из чего складывается стоимость выполнения запроса?	21
4.10	Какие существуют варианты реализации соединений в СУБД? Как установить тип соединения на основе плана запроса?.....	21
4.11	Какие ограничения существуют для различных типов соединений?.....	21
4.12	Как на основе плана выполнения запроса можно распараллеливать выполнение его операций?	22
4.13	В каких случаях менее эффективный на первый взгляд план запроса может привести к большей производительности при использовании вместе с	

секционированием и наличии процессора с несколькими ядрами/нескольких процессоров?	22
5 Выводы	23
Список используемых источников	24

1 ФОРМУЛИРОВКА ЗАДАНИЯ

Цель лабораторной работы — получение навыков повышения производительности, оптимизации выполнения запросов и анализа планов запросов в СУБД.

Работа выполняется над базой данных, разработанной при выполнении лабораторной работы №1. Для оптимизации используются запросы к данным согласно варианту предметной области. В рамках работы необходимо заполнить тестовую базу данных, получить планы выполнения запросов, определить узкие места, предложить индексы, повторно снять планы выполнения и сравнить время выполнения запросов до и после оптимизации.

В качестве СУБД использовалась PostgreSQL 16.13, запущенная в Docker-контейнере в среде WSL Ubuntu. Для выполнения запросов и снятия планов использовался клиент `psql`. Экспериментальная база данных имела имя `subd_lab5`.

2 ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

2.1 Планы выполнения запросов

План выполнения запроса описывает, какие операции СУБД выполняет для получения результата: чтение таблиц, использование индексов, соединение отношений, сортировку, группировку и агрегацию. В PostgreSQL для анализа планов применяется команда EXPLAIN. При использовании EXPLAIN ANALYZE запрос не только планируется, но и реально выполняется, поэтому в плане отображается фактическое время выполнения операций и количество обработанных строк.

Наиболее важными элементами плана являются тип доступа к данным, количество строк, отбрасываемых фильтрами, вид соединения таблиц, стоимость сортировки и итоговое время выполнения. Последовательное сканирование таблицы может быть оправдано для небольших таблиц или низкоселективных условий, однако при выборке небольшой части данных из большой таблицы оно часто становится узким местом.

2.2 Индексы и оптимизация запросов

Индекс представляет собой дополнительную физическую структуру данных, ускоряющую поиск строк по значениям одного или нескольких атрибутов. Индексы особенно эффективны, когда условия запроса обладают высокой селективностью, то есть позволяют отобрать относительно небольшую часть строк таблицы.

В PostgreSQL могут применяться составные индексы, частичные индексы и покрывающие индексы с INCLUDE-атрибутами. Составной индекс строится по нескольким столбцам и эффективен, если порядок столбцов соответствует условиям фильтрации. Частичный индекс содержит только строки, удовлетворяющие заданному предикату, что уменьшает размер индексной структуры. Индексное чтение типа Index Only Scan позволяет получить

необходимые данные из индекса без обращения к строкам таблицы, если все требуемые атрибуты доступны из индекса.

3 РЕЗУЛЬТАТЫ РАБОТЫ

3.1 Подготовка экспериментальной базы данных

В ходе работы использовалась схема базы данных, разработанная в лабораторной работе №1 для варианта №8. База данных предназначена для хранения сведений о документах организации и журнале операций, выполняемых с этими документами.

В базе данных использовались два отношения: `documents` и `document_operations`. Таблица `documents` хранит сведения о документах организации: номер документа, тип документа, ответственного сотрудника, содержимое документа в формате `jsonb`, внутреннее подразделение, дату ввода в действие, дату окончания действия и степень секретности. Таблица `document_operations` хранит журнал операций: ФИО сотрудника, номер документа, дату изменения и описание операции.

Таблица 1 – Разработанные отношения базы данных

Отношение	Назначение	Основные атрибуты	Ключи и ограничения
<code>documents</code>	Хранение сведений о документах организации	<code>document_number</code> , <code>document_type</code> , <code>responsible_employee</code> , <code>document_content</code> , <code>internal_department</code> , <code>effective_from</code> , <code>effective_until</code> , <code>secrecy_level</code>	<code>document_number</code> — PK; <code>document_type</code> , <code>responsible_employee</code> , <code>internal_department</code> , <code>effective_from</code> — АК; проверки дат и допустимых значений <code>secrecy_level</code>
<code>document_operations</code>	Хранение операций, выполненных с документами	<code>employee_name</code> , <code>document_number</code> , <code>change_date</code> , <code>operation_description</code>	<code>employee_name</code> , <code>document_number</code> , <code>change_date</code> — составной PK; <code>document_number</code> — FK на <code>documents(document_number)</code>

Листинг 1 – Создание основных таблиц базы данных

```
CREATE TABLE documents
(
    document_number    text PRIMARY KEY,
    document_type      text NOT NULL,
    responsible_employee text NOT NULL,
    document_content    jsonb NOT NULL,
```

```

internal_department    text NOT NULL,
effective_from         date NOT NULL,
effective_until        date,
secrecy_level          text,
CONSTRAINT uq_documents_alternative_key UNIQUE
(
    document_type,
    responsible_employee,
    internal_department,
    effective_from
),
CONSTRAINT chk_documents_dates CHECK
(
    effective_until IS NULL OR effective_until >= effective_from
),
CONSTRAINT chk_documents_secrecy CHECK
(
    secrecy_level IS NULL OR secrecy_level IN ('ДСП', 'секретно')
)
);

CREATE TABLE document_operations
(
    employee_name        text NOT NULL,
    document_number      text NOT NULL REFERENCES documents(document_number)
                        ON UPDATE CASCADE,
                        ON DELETE CASCADE,
    change_date          timestamp NOT NULL,
    operation_description text NOT NULL,
    CONSTRAINT pk_document_operations PRIMARY KEY
    (
        employee_name,
        document_number,
        change_date
    )
);

```

3.2 Заполнение тестовыми данными

Для проведения эксперимента база данных была заполнена синтетическими тестовыми данными. В таблицу documents было добавлено 100000 записей, в таблицу document_operations — 300000 записей. Для каждого документа было сгенерировано по три операции. При генерации учитывалось распределение по подразделениям, сотрудникам, степеням секретности, датам и типам операций.

Листинг 2 – Фрагмент автоматической генерации тестовых данных

```

INSERT INTO documents
(
    document_number,
    document_type,
    responsible_employee,
    document_content,
    internal_department,
    effective_from,
    effective_until,

```



```

        secrecy_level
    )
SELECT
    'DOC-' || lpad(g::text, 7, '0') AS document_number,
    (ARRAY['Регламент', 'План реагирования', 'Инструкция', 'Модель угроз', 'Служебная записка'])
        [((g * 7 + g / 11) % 5) + 1] AS document_type,
    (ARRAY['Сотрудник_А', 'Сотрудник_Б', 'Сотрудник_В', 'Сотрудник_Г', 'Сотрудник_Д'])
        [((g * 3 + g / 17) % 5) + 1] AS responsible_employee,
    jsonb_build_object('title', 'Тестовый документ ' || g,
        'version', ((g * 13) % 10) + 1,
        'summary', 'Синтетические данные для ЛР5') AS
document_content,
    'Подразделение_' || (((g * 13 + g / 29) % 20) + 1) AS internal_department,
    DATE '2020-01-01' + g::int AS effective_from,
    CASE WHEN g % 17 = 0 THEN NULL
        ELSE DATE '2020-01-01' + g::int + (((g * 31) % 730) + 30)
    END AS effective_until,
    CASE WHEN ((g * 17 + g / 7) % 10) IN (0, 1) THEN 'ДСП'
        WHEN ((g * 17 + g / 7) % 10) = 2 THEN 'секретно'
        ELSE NULL
    END AS secrecy_level
FROM generate_series(1, 100000) AS g;

```

Таблица 2 – Количество записей и размеры таблиц

Отношение	Количество записей	Размер таблицы	Полный размер с индексами
documents	100000	33 MB	54 MB
document_operations	300000	34 MB	58 MB

3.3 Запрос Q1: выборка секретных действующих документов

Запрос Q1 выбирает секретные действующие документы из нескольких подразделений. Он использует фильтрацию по подразделению, степени секретности и сроку действия документа, а также сортировку результата по дате окончания действия и номеру документа.

Листинг 3 – SQL-код запроса Q1

```

SELECT
    document_number,
    document_type,
    responsible_employee,
    internal_department,
    effective_from,
    effective_until,
    secrecy_level
FROM documents
WHERE internal_department IN ('Подразделение_3', 'Подразделение_7',
    'Подразделение_13')
    AND secrecy_level = 'секретно'
    AND effective_until IS NOT NULL

```

```
AND effective_until >= DATE '2024-01-01'
ORDER BY effective_until, document_number;
```

Логический план запроса: выборка строк из отношения documents по условиям secrecy_level, internal_department и effective_until; проекция требуемых атрибутов; сортировка результата по effective_until и document_number.

Листинг 4 – Фрагмент исходного физического плана Q1

```
Sort (actual time=26.412..26.535 rows=2060 loops=1)
  Sort Key: documents.effective_until, documents.document_number
  -> Seq Scan on public.documents
    Filter: effective_until IS NOT NULL
      AND effective_until >= '2024-01-01'
      AND secrecy_level = 'секретно'
      AND internal_department = ANY (...)
    Rows Removed by Filter: 97940
Execution Time: 26.670 ms
```

Узким местом исходного плана является последовательное сканирование всей таблицы documents. PostgreSQL просматривает 100000 строк, после чего отбрасывает 97940 строк фильтром. Так как результирующая выборка составляет небольшую часть таблицы, для запроса целесообразно создать индекс по атрибутам фильтрации.

Таблица 3 – Время выполнения Q1 до и после оптимизации

Запуск	До оптимизации, ms	После оптимизации, ms
1	12.791	2.028
2	10.760	3.337
3	11.604	2.456
4	11.672	1.926
5	10.642	1.927

Листинг 5 – Индекс для оптимизации Q1

```
CREATE INDEX idx_documents_q1_secret_department_until
ON documents
(
  secrecy_level,
  internal_department,
  effective_until,
  document_number
)
INCLUDE
(
  document_type,
  responsible_employee
)
```

```
WHERE secrecy_level IS NOT NULL
AND effective_until IS NOT NULL;
```

Листинг 6 – Фрагмент плана Q1 после оптимизации

```
Bitmap Heap Scan on public.documents
  Recheck Cond: secrecy_level = 'секретно'
                  AND internal_department = ANY (...)
                  AND effective_until >= '2024-01-01'
-> Bitmap Index Scan on idx_documents_q1_secret_department_until
    Index Cond: secrecy_level = 'секретно'
                  AND internal_department = ANY (...)
                  AND effective_until >= '2024-01-01'
Execution Time: 8.878 ms
```

После оптимизации вместо Seq Scan используется Bitmap Index Scan с последующим Bitmap Heap Scan. Среднее время выполнения по пяти тестам уменьшилось с 11.494 ms до 2.335 ms. Коэффициент ускорения составил 4.923, снижение времени выполнения — 79.69 %.

3.4 Запрос Q2: агрегация операций сотрудника за период

Запрос Q2 выполняет соединение журнала операций с таблицей документов и считает количество операций сотрудника за заданный период по ответственным сотрудникам и подразделениям.

Листинг 7 – SQL-код запроса Q2

```
SELECT
  d.responsible_employee,
  d.internal_department,
  count(*) AS operations_count
FROM document_operations AS o
JOIN documents AS d
  ON d.document_number = o.document_number
WHERE o.employee_name = 'Сотрудник_В'
  AND o.change_date >= TIMESTAMP '2024-06-01 00:00:00'
  AND o.change_date <  TIMESTAMP '2024-09-01 00:00:00'
  AND d.secrecy_level IS NOT NULL
GROUP BY
  d.responsible_employee,
  d.internal_department
ORDER BY
  operations_count DESC,
  d.responsible_employee,
  d.internal_department;
```

Логический план запроса: выборка строк из document_operations по сотруднику и диапазону дат; выборка секретных документов из documents; соединение отношений по document_number; группировка по responsible_employee и internal_department; подсчет числа операций; сортировка результата.

Листинг 8 – Фрагмент исходного физического плана Q2

```
Parallel Hash Join
  Hash Cond: d.document_number = o.document_number
    -> Parallel Seq Scan on public.documents d
        Filter: d.secretcy_level IS NOT NULL
    -> Parallel Hash
        -> Parallel Seq Scan on public.document_operations o
            Filter: employee_name = 'Сотрудник В'
                   AND change_date >= '2024-06-01'
                   AND change_date < '2024-09-01'
Execution Time: 34.391 ms
```

Основное узкое место исходного плана — параллельное последовательное сканирование таблицы document_operations с последующим отбрасыванием строк по сотруднику и дате. Также таблица documents сканируется для отбора документов со степенью секретности.

Таблица 4 – Время выполнения Q2 до и после оптимизации

Запуск	До оптимизации, ms	После оптимизации, ms
1	24.745	12.157
2	19.851	9.471
3	19.170	9.680
4	23.652	10.746
5	19.813	10.541

Листинг 9 – Индексы для оптимизации Q2

```
CREATE INDEX idx_operations_q2_employee_date_document
ON document_operations
(
    employee_name,
    change_date,
    document_number
);

CREATE INDEX idx_documents_q2_secret_document_employee_department
ON documents
(
    document_number,
    responsible_employee,
    internal_department
);
```

```
)  
WHERE secrecy_level IS NOT NULL;
```

Листинг 10 – Фрагмент плана Q2 после оптимизации

```
Hash Join  
  Hash Cond: d.document_number = o.document_number  
    -> Index Only Scan using  
idx_documents_q2_secret_document_employee_department  
    on public.documents d  
    Heap Fetches: 0  
  -> Hash  
    -> Index Only Scan using idx_operations_q2_employee_date_document  
    on public.document_operations o  
    Index Cond: employee_name = 'Сотрудник_В'  
                AND change_date >= '2024-06-01'  
                AND change_date < '2024-09-01'  
    Heap Fetches: 0  
Execution Time: 28.945 ms
```

После создания индексов PostgreSQL использует Index Only Scan по обеим таблицам. Операция Hash Join остается, так как запрос требует соединения двух отношений. Среднее время выполнения уменьшилось с 21.446 ms до 10.519 ms. Коэффициент ускорения составил 2.039, снижение времени выполнения — 50.95 %.

3.5 Запрос Q3: поиск документов с изменением срока действия

Запрос Q3 определяет количество несекретных документов по подразделениям, для которых в журнале операций есть операция изменения срока действия за заданный период. Запрос использует EXISTS, то есть проверяет наличие хотя бы одной связанной записи в таблице document_operations.

Листинг 11 – SQL-код запроса Q3

```
SELECT  
  d.internal_department,  
  count(*) AS changed_documents  
FROM documents AS d  
WHERE d.secrecy_level IS NULL  
  AND EXISTS  
  (  
    SELECT 1  
    FROM document_operations AS o  
    WHERE o.document_number = d.document_number  
      AND o.operation_description = 'Изменение срока действия'  
      AND o.change_date >= TIMESTAMP '2024-01-01 00:00:00'  
      AND o.change_date <  TIMESTAMP '2025-01-01 00:00:00'  
  )  
GROUP BY
```

```

d.internal_department
ORDER BY
changed_documents DESC,
d.internal_department;

```

Логический план запроса: выборка несекретных документов; проверка существования связанных операций с описанием «Изменение срока действия» в заданном диапазоне дат; группировка по подразделению; подсчет количества документов; сортировка результата.

Листинг 12 – Фрагмент исходного физического плана Q3

```

Parallel Hash Semi Join
Hash Cond: d.document_number = o.document_number
-> Parallel Seq Scan on public.documents d
    Filter: d.secrecy_level IS NULL
-> Parallel Hash
    -> Parallel Seq Scan on public.document_operations o
        Filter: operation_description = 'Изменение срока действия'
            AND change_date >= '2024-01-01'
            AND change_date < '2025-01-01'
Execution Time: 49.575 ms

```

Узким местом является последовательное чтение таблицы document_operations, где фильтр по описанию операции и диапазону дат применяется после просмотра большого числа строк. Таблица documents также сканируется для отбора несекретных документов.

Таблица 5 – Время выполнения Q3 до и после оптимизации

Запуск	До оптимизации, ms	После оптимизации, ms
1	35.672	20.921
2	43.794	20.259
3	40.446	18.557
4	38.531	19.138
5	45.099	19.197

Листинг 13 – Индексы для оптимизации Q3

```

CREATE INDEX idx_operations_q3_operation_date_document
ON document_operations
(
    operation_description,
    change_date,
    document_number
);

CREATE INDEX idx_documents_q3_public_document_department
ON documents

```

```
(
    document_number,
    internal_department
)
WHERE secrecy_level IS NULL;
```

Листинг 14 – Фрагмент плана Q3 после оптимизации

```
Hash Semi Join
Hash Cond: d.document_number = o.document_number
-> Index Only Scan using idx_documents_q3_public_document_department
    on public.documents d
    Heap Fetches: 0
-> Hash
    -> Index Only Scan using idx_operations_q3_operation_date_document
        on public.document_operations o
        Index Cond: operation_description = 'Изменение срока действия'
                    AND change_date >= '2024-01-01'
                    AND change_date < '2025-01-01'
        Heap Fetches: 0
Execution Time: 39.098 ms
```

После оптимизации PostgreSQL использует Index Only Scan по таблице операций и частичный индекс по несекретным документам. Операция Hash Semi Join соответствует семантике EXISTS. Среднее время выполнения уменьшилось с 40.708 ms до 19.614 ms. Коэффициент ускорения составил 2.075, снижение времени выполнения — 51.82 %.

3.6 Сводные результаты и графики

Таблица 6 – Сводные результаты измерений

Запрос	До оптимизации, ms	После оптимизации, ms	Ускорение	Улучшение, %
Q1	11.494	2.335	4.923	79.69
Q2	21.446	10.519	2.039	50.95
Q3	40.708	19.614	2.075	51.82

Наибольшее ускорение было получено для Q1, так как запрос имеет высокую селективность и использует условия фильтрации, хорошо соответствующие составному частичному индексу. Для Q2 и Q3 ускорение также получено, но оно меньше из-за необходимости соединения таблиц, группировки и сортировки.

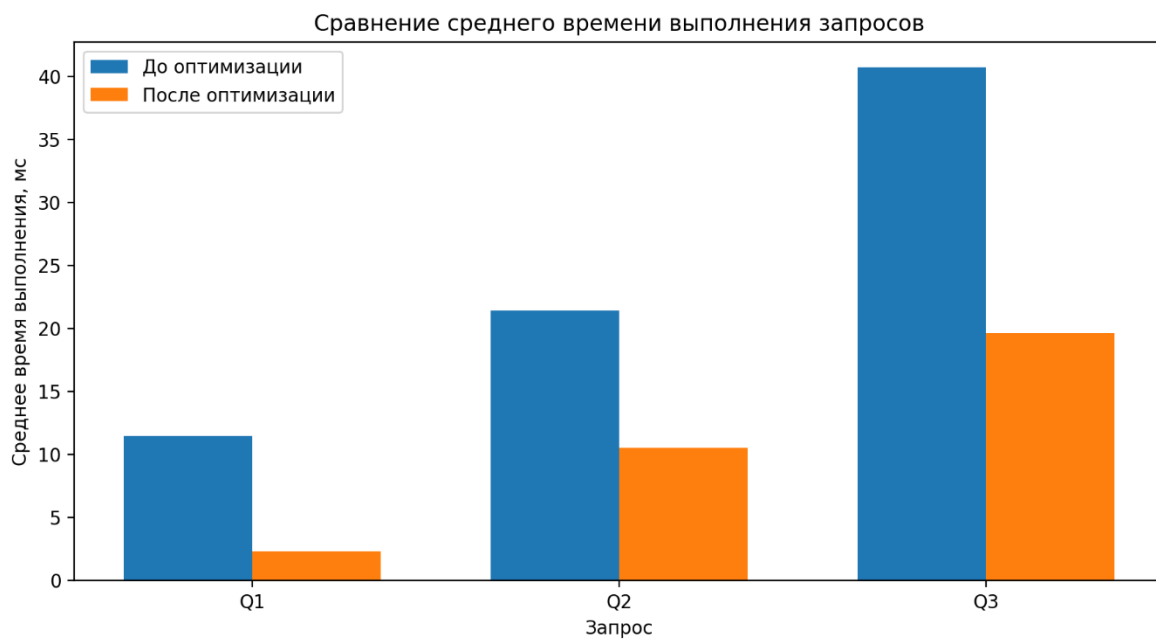


Рисунок 1 – Сравнение среднего времени выполнения запросов

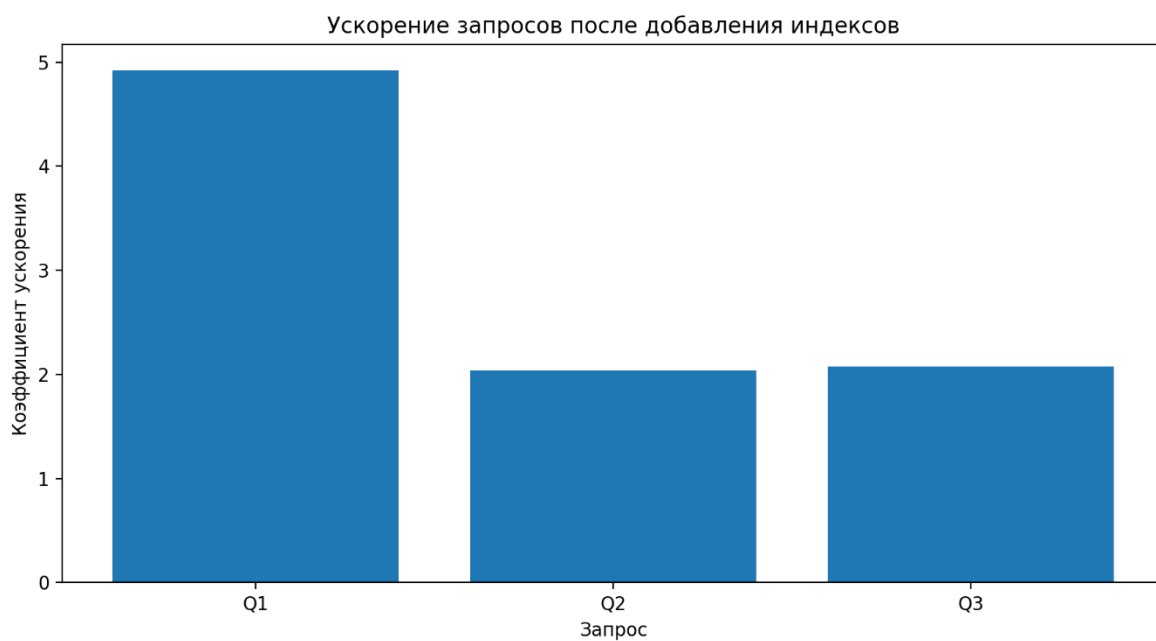


Рисунок 2 – Коэффициент ускорения после добавления индексов

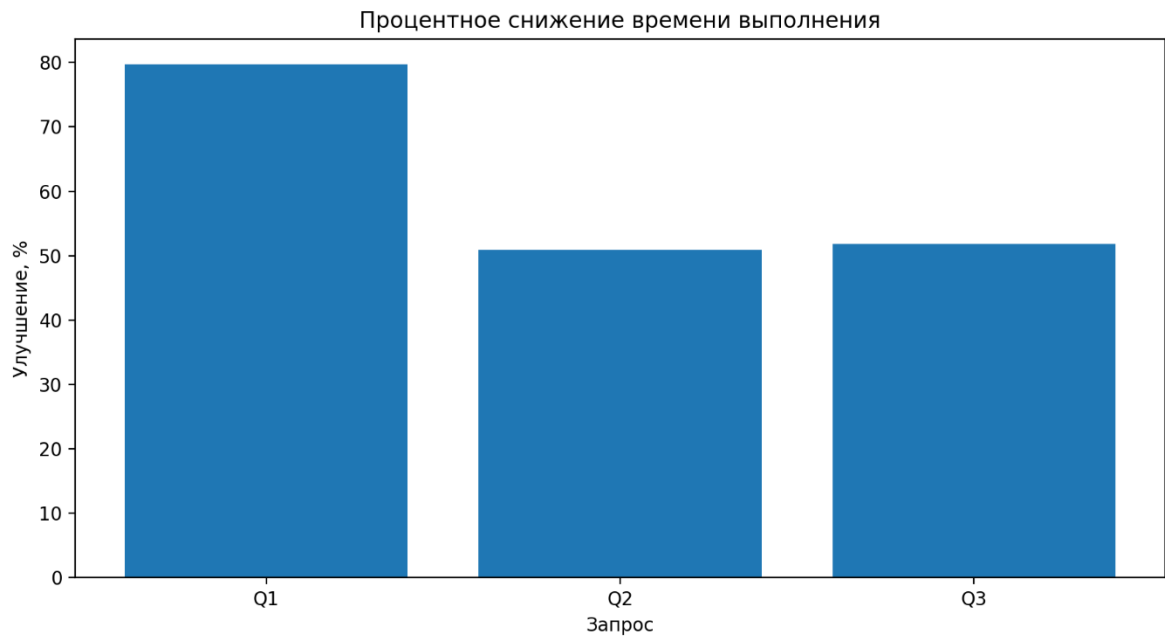


Рисунок 3 – Процентное снижение времени выполнения запросов

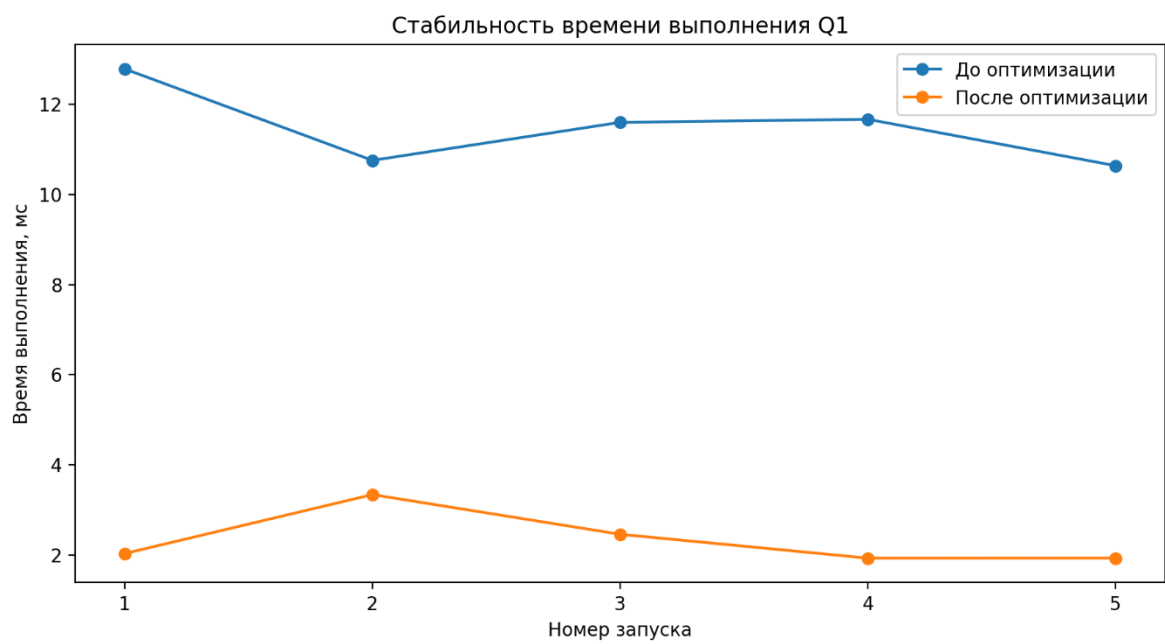


Рисунок 4 – Стабильность времени выполнения запроса Q1

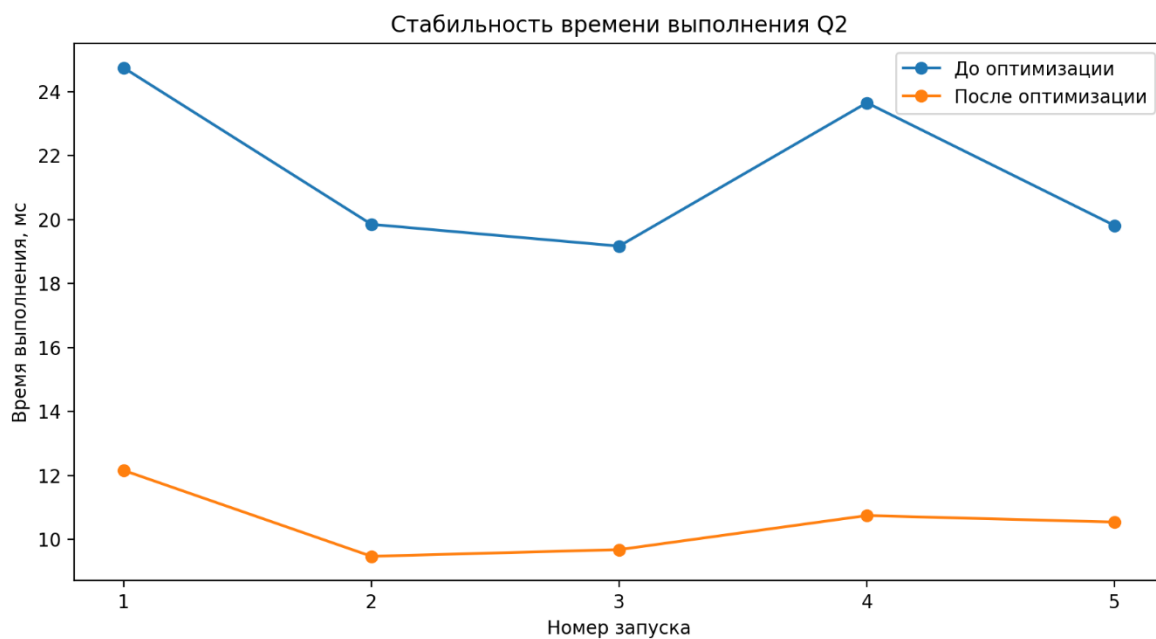


Рисунок 5 – Стабильность времени выполнения запроса Q2

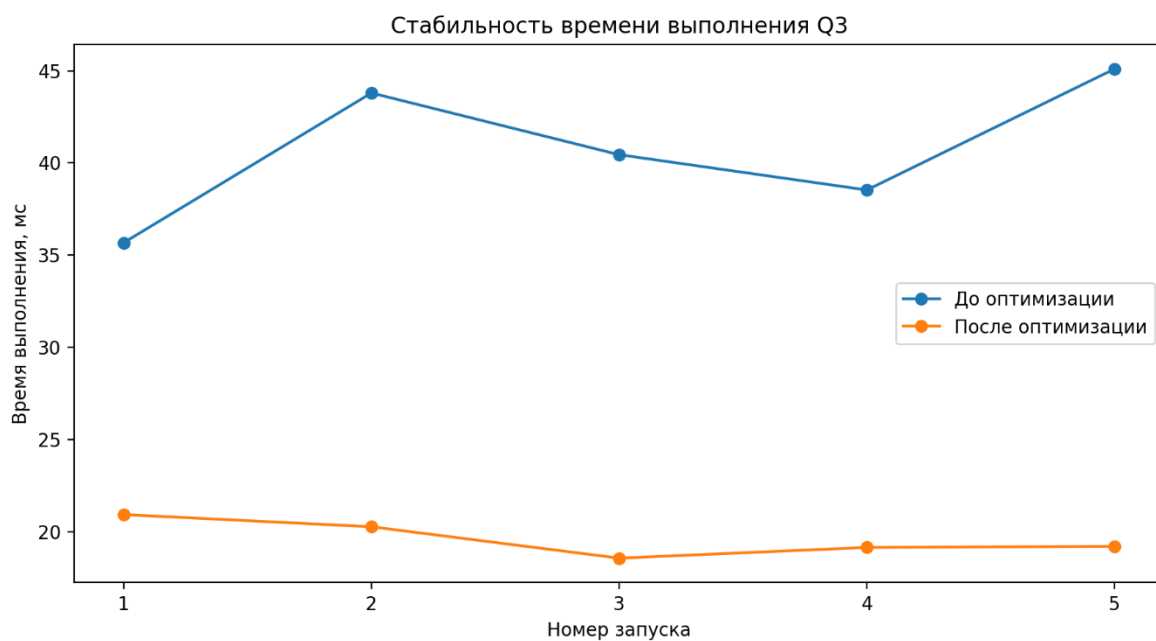


Рисунок 6 – Стабильность времени выполнения запроса Q3

4 ОТВЕТЫ НА КОНТРОЛЬНЫЕ ВОПРОСЫ

4.1 Что такое логический план выполнения запроса, когда он формируется и для чего используется?

Логический план выполнения запроса — это представление запроса в виде операций реляционной алгебры: выборки, проекции, соединения, группировки, сортировки и других операций без привязки к конкретным физическим методам доступа. Он формируется после синтаксического и семантического анализа SQL-запроса и используется оптимизатором как основа для выбора эффективного физического плана.

4.2 Как связаны логический и физический план выполнения запроса?

Логический план описывает, какие операции должны быть выполнены для получения результата, а физический план определяет, как именно они будут выполнены средствами конкретной СУБД. Например, логическая операция соединения может быть реализована физически как Nested Loop Join, Hash Join или Merge Join, а логическая выборка может выполняться через Seq Scan, Index Scan или Bitmap Heap Scan.

4.3 Что такое предварительный и реальный план выполнения запроса? Почему они могут отличаться?

Предварительный план показывает оценку оптимизатора до фактического выполнения запроса. Реальный план, полученный через EXPLAIN ANALYZE, содержит фактические времена, количество строк и число циклов выполнения. Они могут отличаться из-за неточной статистики, неравномерного распределения данных, устаревших оценок селективности, влияния кэша, параллелизма и текущего состояния сервера.

4.4 Как селективность атрибута связана с индексами и производительностью?

Селективность показывает, насколько сильно условие по атрибуту уменьшает количество строк результата. Чем выше селективность, тем эффективнее индекс, потому что СУБД может быстро найти небольшое число строк. Если условие возвращает большую часть таблицы, последовательное сканирование может быть выгоднее индексного доступа.

4.5 Когда применение индексов эффективно?

Индексы эффективны при выборке небольшой части строк из большой таблицы, при соединениях по индексируемым ключам, при фильтрации по диапазонам и при сортировке, если порядок индекса соответствует ORDER BY. Индекс также эффективен, если он покрывает запрос и позволяет выполнить Index Only Scan.

4.6 Как порядок полей в составном индексе влияет на производительность различных запросов и применение индекса?

Порядок полей в составном индексе определяет, какие условия запроса могут быть использованы для быстрого поиска. Обычно первыми размещают поля с равенством и высокой селективностью, затем диапазонные поля и поля сортировки. Если запрос не использует начальные поля составного индекса, индекс может применяться хуже или не применяться вообще.

4.7 Какая операция указывается в физическом плане запроса?

В физическом плане указываются конкретные операции, выбранные СУБД: Seq Scan, Index Scan, Index Only Scan, Bitmap Index Scan, Bitmap Heap Scan, Nested Loop, Hash Join, Merge Join, Sort, Aggregate, HashAggregate, Gather, Gather Merge и другие узлы выполнения.

4.8 Как прочитать текстовый план выполнения запроса от первой операции к последней?

Текстовый план читается снизу вверх и от наиболее вложенных узлов к корневому узлу. Сначала выполняются операции чтения таблиц и индексов, затем фильтрация, соединения, группировка, сортировка и финальная выдача результата. Отступы в плане показывают вложенность операций.

4.9 Из чего складывается стоимость выполнения запроса?

Стоимость складывается из чтения страниц таблиц и индексов, CPU-затрат на обработку строк, фильтрации, соединений, сортировки, группировки, агрегации, передачи промежуточных результатов, параллельного выполнения и использования памяти. В PostgreSQL стоимость является относительной оценкой оптимизатора, а не временем в миллисекундах.

4.10 Какие существуют варианты реализации соединений в СУБД? Как установить тип соединения на основе плана запроса?

Основные физические варианты соединений — Nested Loop Join, Hash Join и Merge Join. Тип соединения определяется непосредственно по узлу плана выполнения. Например, строки Nested Loop, Hash Join или Merge Join указывают, каким способом СУБД реализовала логическую операцию JOIN.

4.11 Какие ограничения существуют для различных типов соединений?

Nested Loop Join эффективен для небольших наборов или при наличии индекса на внутренней таблице, но плохо работает при больших объемах без индексов. Hash Join требует памяти для построения хеш-таблицы и обычно применяется для равенств. Merge Join требует отсортированных входов или затрат на сортировку, зато эффективен для больших предварительно упорядоченных наборов.

4.12 Как на основе плана выполнения запроса можно распараллеливать выполнение его операций?

По плану можно определить независимые или дорогостоящие операции чтения, соединения и агрегации, которые могут выполняться параллельно. В PostgreSQL параллелизм отображается узлами Parallel Seq Scan, Parallel Hash Join, Gather и Gather Merge. Если такие узлы отсутствуют, можно анализировать причины: малый объем данных, неподходящие настройки `parallel_workers`, высокая стоимость запуска параллелизма или невозможность распараллеливания конкретной операции.

4.13 В каких случаях менее эффективный на первый взгляд план запроса может привести к большей производительности при использовании вместе с секционированием и наличии процессора с несколькими ядрами/нескольких процессоров?

Менее эффективный по локальной оценке план может оказаться быстрее, если он хорошо распараллеливается или позволяет независимо обрабатывать секции таблицы. Например, несколько последовательных сканирований секций на разных ядрах или дисках могут выполняться быстрее, чем один индексный доступ с большим числом случайных обращений. Поэтому итоговая производительность зависит не только от стоимости отдельной операции, но и от параллелизма, секционирования, размещения данных и характера ввода-вывода.

5 ВЫВОДЫ

В ходе лабораторной работы была подготовлена экспериментальная база данных PostgreSQL для предметной области документов организации. В таблицу documents было добавлено 100000 записей, в таблицу document_operations — 300000 записей. Для анализа производительности были разработаны три запроса, включающие фильтрацию, соединение таблиц, EXISTS, группировку и сортировку.

Для каждого запроса были получены исходные планы выполнения с помощью EXPLAIN ANALYZE. В исходных планах были обнаружены операции последовательного сканирования таблиц documents и document_operations, что приводило к чтению большого количества строк и последующему отбрасыванию значительной части данных по условиям фильтрации.

Для оптимизации были созданы составные и частичные индексы, соответствующие условиям фильтрации и соединения. После добавления индексов PostgreSQL стал использовать Bitmap Index Scan, Bitmap Heap Scan и Index Only Scan. Это позволило снизить среднее время выполнения всех исследованных запросов: Q1 был ускорен в 4.923 раза, Q2 — в 2.039 раза, Q3 — в 2.075 раза.

Полученные результаты показывают, что анализ планов выполнения позволяет выявлять узкие места запросов, а правильно подобранные индексы существенно повышают производительность операций выборки. При этом эффективность индекса зависит от селективности условий, порядка полей в составном индексе, структуры запроса и стоимости последующих операций соединения, группировки и сортировки.

СПИСОК ИСПОЛЬЗУЕМЫХ ИСТОЧНИКОВ

1. Дейт, К. Дж. Введение в системы баз данных, 8-е издание. — М.: Вильямс, 2005. — 1328 с.
2. Моргунов, Е. П. PostgreSQL. Основы языка SQL : учебное пособие / Е. П. Моргунов ; под ред. Е. В. Рогова, П. В. Лузанова. — Санкт-Петербург : БХВ-Петербург, 2018. — 336 с.
3. Полтавцева, М. А. Практикум по построению защищенных баз данных: учебное пособие / М. А. Полтавцева [и др.]. — Санкт-Петербург : ПОЛИТЕХ-ПРЕСС, 2023. — 182 с.
4. Рогов, Е. В. PostgreSQL 15 изнутри / Е. В. Рогов. — Москва : ДМК Пресс, 2023. — 662 с.
5. The PostgreSQL Global Development Group. PostgreSQL Documentation: EXPLAIN. — Режим доступа: <https://www.postgresql.org/docs/current/sql-explain.html>
6. The PostgreSQL Global Development Group. PostgreSQL Documentation: Indexes. — Режим доступа: <https://www.postgresql.org/docs/current/indexes.html>